

13. *Skip List*

–*Class Tweaks*

```
class Node {
    public: // Must explicitly declare public so SkipList can access
data
    int key;
    Node **forward;

    // Constructor handles node memory allocation via 'new'
    Node(int k, int level) {
        key = k;
        forward = new Node*[level + 1];
        for (int i = 0; i <= level; i++) {
            forward[i] = nullptr;
        }
    }
}
```

*forward = new Node * [level + 1];*

is same as:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level +
1));
```

*When the compiler reads **new Node * [level + 1];**, it reads it from left to right like a set of instructions:*

1. **new (The Action):** *The compiler tells the OS, "Go to the Heap and carve out a contiguous block of memory."*

2. **Node * (The Data Type):** This answers the question: What is going to be stored in this memory block? It is storing single pointers (**Node ***). Since every single pointer is 8 bytes.
3. **[level + 1] (The Size):** This is how many single pointers we need. If the node's random level is 3, it calculates $3 + 1 = 4$ slots.
4. **The Math:** The compiler automatically calculates **4 slots $\times$ 8 bytes = 32 bytes**.
5. **forward = (The Assignment):** Finally, **new** returns the base memory address of that 32 – byte block (**e.g., 0x8000**) and stores it inside our double pointer variable, **forward**.

```
void clearList() {
    Node *current = header->forward[0];

    // Traverse level 0 and delete every node
    while (current != nullptr) {
        Node *next = current->forward[0];
        delete current; // Calls ~Node() automatically
        current = next;
    }

    // Reset the header to empty
    for (int i = 0; i <= MAX_LEVEL; i++) {
        header->forward[i] = nullptr;
    }
    level = 0;

    cout << "Memory successfully freed. Skip List cleared." <<
endl;
}
```

```
~Node() {
    delete[] forward;
}
```

*The Two – Step Mechanism of **delete***

*When the CPU executes **delete current**;, the C++ runtime performs two distinct actions behind the scenes:*

```
delete current;  
├── 1. Call Destructor (~Node()) → Cleans up internal resources (e.g., delete[] forward)  
└── 2. Free Memory (operator delete) → Releases the Node's own memory back to the Heap
```

Now,

```
~Node() {  
    delete[] forward;  
}
```

is same as:

```
free(current->forward);
```

Now freeing memory by delete keyword:

is same as:

```
free(current);
```
